

Esercitazione – Pianificazione dei processi della CPU

Introduzione

In questa esercitazione ho confrontato tre diversi metodi di gestione dei processi per capire quale metodo permette di utilizzare meglio la CPU. I processi considerati sono quattro (P1, P2, P3 e P4) e per ciascuno sono indicati il tempo di esecuzione, l'eventuale tempo di attesa dovuto alle operazioni di input/output e, nel caso di P1, un secondo intervallo di esecuzione.

I tre scenari analizzati sono:

- sistema mono-tasking;
- sistema multi-tasking;
- sistema time-sharing.

Processo	Esecuzione	Attesa I/O	Seconda esecuzione
P1	3 s	1 s	1 s
P2	1 s	2 s	-
P3	2 s	-	-
P4	4 s	1 s	-

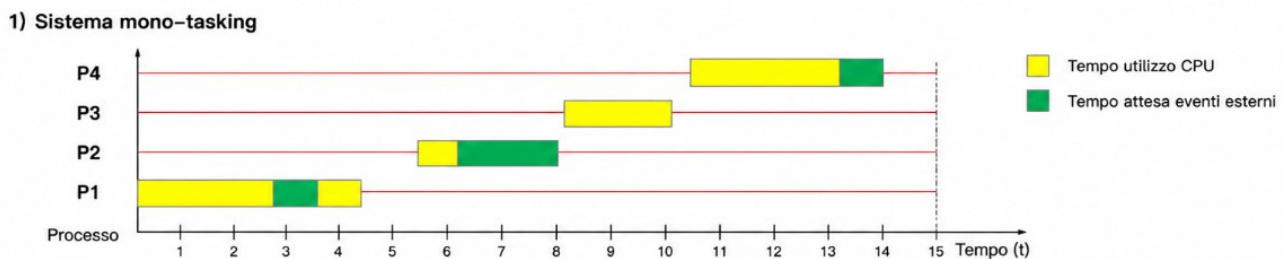
Sistema mono-tasking

Nel sistema mono-tasking viene eseguito un solo processo alla volta.

Quando un processo entra nella fase di attesa per un'operazione di input/output, la CPU rimane inutilizzata fino a quando il processo non può riprendere l'esecuzione. Solo dopo la sua conclusione viene avviato il processo successivo.

Questo metodo è semplice ma poco efficiente, perché durante i tempi di attesa la CPU rimane ferma anche se ci sono altri processi pronti per essere eseguiti.

Figura 1 – Diagramma del sistema mono-tasking



Osservazione: dal diagramma si nota che quando il processo entra nella fase di attesa per l'input/output, la CPU rimane inutilizzata fino a quando il processo non può riprendere l'esecuzione.

Sistema multi-tasking

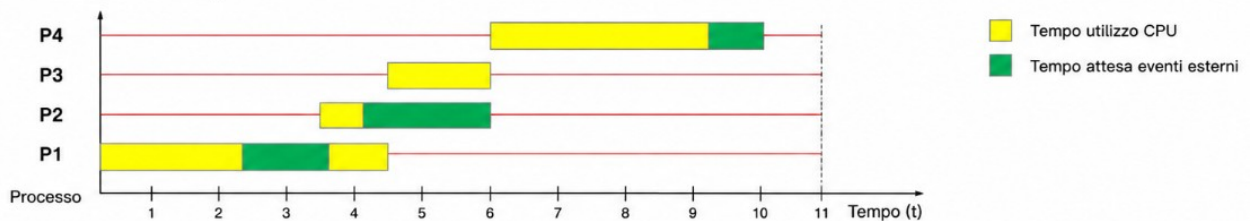
Nel sistema multi-tasking, quando un processo entra in attesa di un evento esterno (ad esempio un'operazione di I/O), il sistema assegna immediatamente la CPU ad un altro processo pronto.

In questo modo il processore continua a lavorare quasi continuamente, riducendo i tempi morti e migliorando l'utilizzo delle risorse.

Rispetto al sistema precedente, la CPU rimane occupata per gran parte del tempo e i processi vengono completati in modo più efficiente.

Figura 2 – Diagramma del sistema multi-tasking

2) Sistema multi-tasking



Osservazione: durante l'attesa di un processo la CPU viene subito assegnata ad un altro processo, riducendo i tempi morti.

Sistema time-sharing

Nel sistema time-sharing la CPU viene condivisa tra tutti i processi attraverso un intervallo di tempo fisso, chiamato **quanto di tempo (time slice)**.

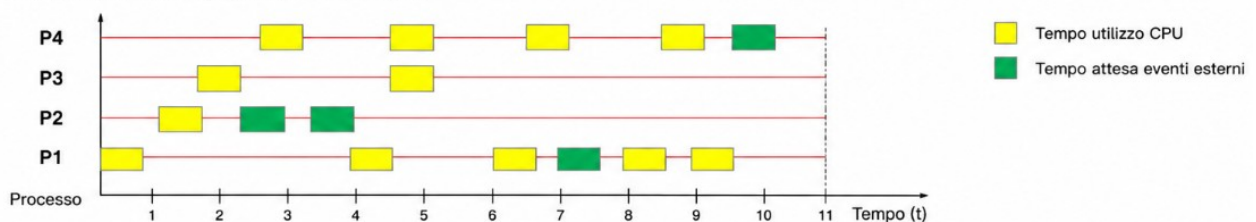
In questo esercizio si considera un quanto pari a **1 secondo**.

Ogni processo utilizza la CPU per un secondo; allo scadere del quanto, se non ha terminato, viene sospeso e rimesso in coda permettendo al processo successivo di essere eseguito.

Questo metodo rende il sistema molto più reattivo perché tutti i processi ricevono rapidamente del tempo di CPU, anche se richiedono tempi di esecuzione lunghi.

Figura 3 – Diagramma del sistema time-sharing

3) Sistema time-sharing (quanto = 1 secondo)



Osservazione: ogni processo utilizza la CPU solo per il tempo previsto dal quanto e poi lascia spazio al processo successivo, rendendo il sistema più reattivo.

Esercizio facoltativo – Round Robin

Per il secondo esercizio ho applicato l'algorithmo **Round Robin** utilizzando un **time slice di 12 millisecondi**.

Nella prima tabella è riportata la sequenza con cui i processi vengono eseguiti e la composizione della coda al termine di ogni quanto di tempo.

Successivamente sono stati calcolati il tempo di turnaround e il tempo medio di attesa di ciascun processo.

Figura 4 – Sequenza di esecuzione Round Robin

P	t ₀	T _x
P1	0	14
P2	30	16
P3	6	40
P4	46	26
P5	22	28

time slice	Inizio	Fine	Processo	Coda a fine time slice
1	0	12	P1	P3 (6*)
2	12	24	P3	P1 (12) P5 (22*)
3	24	26	P1 → FINE	P5 (22) P3 (24)
4	26	38	P5	P3 (24) P2 (30*)
5	38	50	P3	P2 (30) P5 (38) P4 (46*)
6	50	62	P2	P5 (38) P4 (46) P3 (50)
7	62	74	P5	P4 (46) P3 (50) P2 (62)
8	74	86	P4	P3 (50) P2 (62) P5 (74)
9	86	98	P3	P2 (62) P5 (74) P4 (86)
10	98	102	P2 → FINE	P5 (74) P4 (86) P3 (98)
11	102	106	P5 → FINE	P4 (86) P3 (98)
12	106	118	P4	P3 (98)
13	118	122	P3 → FINE	P4 (118)
14	122	124	P4 → FINE	

La tabella seguente riporta invece il calcolo dei tempi di turnaround e dei tempi medi di attesa ottenuti con l'algoritmo Round Robin.

Figura 5 – Calcolo dei tempi di turnaround e di attesa

time slice	Inizio	Fine	Processo		t_0	T_x	t_f	turnaroud [$T_t = t_f - t_0$]	attesa [$T_a = T_t - T_x$]
1	0	12	P1						
2	12	24	P3	P1	0	14	26	26	12
3	24	26	P1 → FINE	P2	30	16	102	72	56
4	26	38	P5	P3	6	40	122	116	76
5	38	50	P3	P4	46	26	124	78	52
6	50	62	P2	P5	22	28	106	84	56
7	62	74	P5	TOT				376	252
8	74	86	P4	MEDIA [TOT/5]				75,2	50,4
9	86	98	P3						
10	98	102	P2 → FINE						
11	102	106	P5 → FINE						
12	106	118	P4						
13	118	122	P3 → FINE						
14	122	124	P4 → FINE						

Considerazioni finali

Tra i tre sistemi analizzati, il mono-tasking è quello meno efficiente perché durante le operazioni di input/output il processore rimane inattivo.

Il multi-tasking migliora notevolmente la situazione, sfruttando la CPU anche quando un processo è in attesa.

Il time-sharing rappresenta un'ulteriore evoluzione, poiché divide il tempo della CPU tra tutti i processi e impedisce che uno solo occupi il processore per troppo tempo. In questo modo il sistema risulta più equilibrato e tutti i processi possono iniziare l'esecuzione senza dover attendere la fine completa di quelli precedenti.

Per questo motivo i sistemi operativi moderni utilizzano tecniche di multi-tasking e time-sharing, che consentono un utilizzo più efficiente della CPU e una migliore gestione dei processi.

Questa esercitazione mi ha aiutato a comprendere meglio il funzionamento delle principali tecniche di pianificazione dei processi e le differenze tra i vari metodi di scheduling.